

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

ДЕНЬ 13

Тема: «Разработка высоконагруженного сервисного слоя для системы
управления бронированиями»

Цель работы:

Сформировать у обучающихся навыки **проектирования и реализации** сервисного слоя с использованием архитектурных паттернов, обеспечивающих слабую связанность, тестируемость и расширяемость.

Выполнила:
Позднякова Марина,
Студент 24-ИС

2026г.

день 13/

- | — main.py ← ТВОЙ ФАЙЛ (ЭТОТ)
- | — src/
 - | | — __init__.py ← ДОЛЖЕН БЫТЬ (пустой)
 - | | — domain/
 - | | | — __init__.py ← ДОЛЖЕН БЫТЬ (пустой)
 - | | | — models.py
 - | | | — exceptions.py
 - | | — dto/
 - | | | — __init__.py ← ДОЛЖЕН БЫТЬ (пустой)
 - | | | — booking_dto.py
 - | | | — room_dto.py
 - | | — repositories/
 - | | | — __init__.py ← ДОЛЖЕН БЫТЬ (пустой)
 - | | | — base.py
 - | | | — hotel_repo.py
 - | | | — room_repo.py
 - | | | — booking_repo.py
 - | | — services/
 - | | | — __init__.py ← ДОЛЖЕН БЫТЬ (пустой)
 - | | | — booking_service.py
 - | | | — pricing_service.py
 - | | — observers/
 - | | | — __init__.py ← ДОЛЖЕН БЫТЬ (пустой)
 - | | | — price_observer.py
 - | | — uow/
 - | | | — __init__.py ← ДОЛЖЕН БЫТЬ (пустой)
 - | | | — unit_of_work.py
- | — requirements.txt

Ответ. Все работает отлично

```
=====
[icon] СИСТЕМА БРОНИРОВАНИЯ ОТЕЛЕЙ
=====

[check] Unit of Work создан
[check] Добавлен отель: Гранд Отель (ID: 1)
[check] Добавлен номер: 101 (ID: 1)
[check] Добавлен номер: 102 (ID: 2)

[check] Сервисы созданы

=====
[icon] СОЗДАНИЕ БРОНИРОВАНИЯ
=====

[check] Бронирование создано!
ID: 1
Гость: Иван Петров
Номер: 1
Даты: 2026-07-15 - 2026-07-20
Сумма: 675.0 руб.
Статус: BookingStatusDTO.PENDING

=====
[icon] ПОИСК ДОСТУПНЫХ НОМЕРОВ
=====

Найдено доступных номеров: 1
  Номер 102 (ID: 2)
    Вместимость: 4 чел.
    Цена за ночь: 150.0 руб.
    Итого: 1125.0 руб.

=====
[icon] ОТМЕНА БРОНИРОВАНИЯ
=====

[check] Бронирование #1 отменено

[icon] Доступных номеров после отмены: 2

=====
[icon] ИТОГОВАЯ СТАТИСТИКА
=====

  Отелей: 1
  Номеров: 2
  Бронирований: 1

=====
[check] РАБОТА ЗАВЕРШЕНА
=====
|
```

контрольные вопросы:

1. Какая основная цель выделения сервисного слоя?

Основная цель выделения сервисного слоя — инкапсуляция бизнес-логики в отдельном слое, чтобы контроллеры занимались только обработкой HTTP-запросов, а вся логика приложения была изолирована, тестируема и переиспользуема независимо от интерфейса.

2. Чем отличается DTO от Domain Model?

DTO (Data Transfer Object) — это объект для передачи данных между слоями, который содержит только поля и валидацию, но не содержит бизнес-логики, в то время как Domain Model — это бизнес-сущность, которая инкапсулирует поведение и правила предметной области.

3. Что такое Unit of Work и зачем он нужен?

Unit of Work — это паттерн, который управляет транзакциями и отслеживает изменения объектов в рамках одной бизнес-операции, обеспечивая атомарность: либо все изменения сохраняются через commit, либо все откатываются через rollback при ошибке.

4. В чем разница между паттернами Repository и Service?

Repository отвечает только за доступ к данным и CRUD-операции, а Service содержит бизнес-логику, использует репозитории для получения данных, применяет правила и управляет транзакциями через Unit of Work.

5. Какую проблему решает внедрение зависимостей (DI)?

Внедрение зависимостей решает проблему жёсткой связанности компонентов, позволяя передавать зависимости через конструктор, что упрощает тестирование с использованием мок-объектов и даёт возможность легко заменять реализации, например, переключаться с in-memory репозитория на реальную базу данных.

6. Почему нельзя использовать бизнес-логику в контроллерах?

Бизнес-логику нельзя размещать в контроллерах, потому что это приводит к дублированию кода, усложняет тестирование, делает логику привязанной к HTTP-протоколу и мешает переиспользовать её в других интерфейсах, например, в CLI-командах или очередях сообщений.

7. Какие типы исключений должны быть в сервисном слое?

В сервисном слое должны быть свои кастомные исключения, наследуемые от базового DomainError, например, RoomNotFoundError, BookingConflictError, InvalidDatesError, которые отражают бизнес-ошибки и позволяют обрабатывать их на уровне выше, не смешивая с техническими исключениями.

8. Как протестировать сервис без подключения к реальной БД?

Сервис тестируется с использованием мок-объектов (Mock), которые заменяют репозитории: мы создаём фиктивные данные, подменяем реальные репозитории на моки, проверяем, что сервис вызывает нужные методы с правильными аргументами и возвращает ожидаемые результаты, при этом реальная база данных не используется.

Вывод о проделанной работе

В ходе выполнения практического задания я спроектировала и реализовала сервисный слой для системы управления бронированиями отелей с использованием паттернов Repository, Service Layer, DTO и Unit of Work. Мною были разработаны все необходимые компоненты: доменные модели, DTO для валидации данных, in-memory репозитории и сервисы с бизнес-логикой, включая динамическое ценообразование и проверку доступности номеров. Я реализовала паттерн Observer для уведомлений об изменении цены и настроила обработку бизнес-исключений, что обеспечило корректную работу системы при создании и отмене бронирований. В ходе отладки я успешно исправила ошибки импорта модулей и предупреждения Pydantic, связанные с переходом на версию 2.0, и протестировала работу всех компонентов через main.py. В результате создана архитектурно правильная, тестируемая и расширяемая система, в которой бизнес-логика изолирована от контроллеров и может быть переиспользована в различных интерфейсах. Данная работа позволила мне закрепить навыки проектирования серверных приложений и применения архитектурных паттернов на практике.